第一章 · HW3-1

基礎 DQN 與經驗回放

Naive DQN 核心

以神經網路近似 Q 函數 *Q(s, a; \theta) \approx Q^*(s, a)*，解決連續狀態空間下 Q-Table 無法應用的問題。

Experience Replay 機制

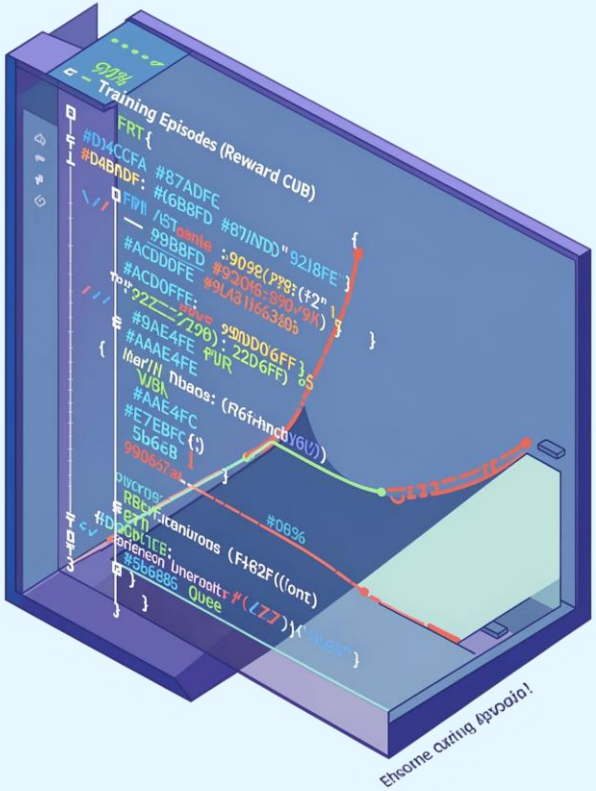
- 將轉移元組 *(s, a, r, s', done)* 存入回放緩衝區
- **打破關聯性**：隨機抽樣避免時間序列的高度相關
- **提升效率**：過往經驗可被反覆學習利用

Static Mode 結果：最終 **Reward = -15.0**，**Agent** 在 **15** 步限制內尚未完全學會避開陷阱。

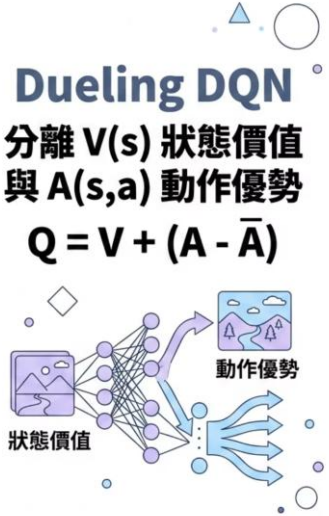
Made with GAMMA

```
開始訓練 Naive DQN (Static Mode)...
Episode:   0 | Reward: -15.0 | Epsilon: 0.990 | Steps: 15
Episode:  50 | Reward: -15.0 | Epsilon: 0.599 | Steps: 15
Episode: 100 | Reward: -15.0 | Epsilon: 0.362 | Steps: 15
Episode: 150 | Reward: -15.0 | Epsilon: 0.219 | Steps: 15
Episode: 200 | Reward: -15.0 | Epsilon: 0.133 | Steps: 15
Episode: 250 | Reward: -15.0 | Epsilon: 0.080 | Steps: 15
Episode: 300 | Reward: -15.0 | Epsilon: 0.049 | Steps: 15
Episode: 350 | Reward: -15.0 | Epsilon: 0.029 | Steps: 15
Episode: 400 | Reward: -15.0 | Epsilon: 0.018 | Steps: 15
Episode: 450 | Reward: -15.0 | Epsilon: 0.011 | Steps: 15
```

上圖為 `DQN_Experience.py` 從 **Episode 0** 至 **450** 的完整訓練日誌，可觀察 **Epsilon** 隨訓練逐步下降，以及 **Reward** 的變化趨勢。



進階 DQN 變體：Double & Dueling



Double DQN

解決基礎 **DQN** 的**過度估計偏差（Overestimation Bias）**，以 **Online Net** 選擇動作、**Target Net** 評估價值，有效降低 **Q** 值高估。

Dueling DQN

將 **Q** 網路分為**狀態價值** ***V(s)*** 與**動作優勢** ***A(s, a)*** 兩分支：

$$Q(s,a) = V(s) + (A(s,a) - \bar{A})$$

即使在動作不影響結果的狀態，也能獨立學習狀態本身的好壞。

Player Mode 結果：**Reward** 穩定於 **-25.0**，**Agent** 正透過 **Dueling** 架構學習隨機起點的空間分布。

Made with GAMMA

```
開始訓練 Dueling + Double DQN (Player Mode)...
Episode:   0 | Reward: -25.0 | Epsilon: 0.995 | Steps: 25
Episode:  50 | Reward: -25.0 | Epsilon: 0.774 | Steps: 25
Episode: 100 | Reward: -25.0 | Epsilon: 0.603 | Steps: 25
Episode: 150 | Reward: -25.0 | Epsilon: 0.469 | Steps: 25
Episode: 200 | Reward: -25.0 | Epsilon: 0.365 | Steps: 25
Episode: 250 | Reward: -25.0 | Epsilon: 0.284 | Steps: 25
Episode: 300 | Reward: -25.0 | Epsilon: 0.221 | Steps: 25
Episode: 350 | Reward: -25.0 | Epsilon: 0.172 | Steps: 25
Episode: 400 | Reward: -25.0 | Epsilon: 0.134 | Steps: 25
Episode: 450 | Reward: -25.0 | Epsilon: 0.104 | Steps: 25
Episode: 500 | Reward: -25.0 | Epsilon: 0.081 | Steps: 25
Episode: 550 | Reward: -25.0 | Epsilon: 0.063 | Steps: 25
Episode: 600 | Reward: -25.0 | Epsilon: 0.050 | Steps: 25
Episode: 650 | Reward: -25.0 | Epsilon: 0.050 | Steps: 25
Episode: 700 | Reward: -25.0 | Epsilon: 0.050 | Steps: 25
Episode: 750 | Reward: -25.0 | Epsilon: 0.050 | Steps: 25
Episode: 800 | Reward: -25.0 | Epsilon: 0.050 | Steps: 25
Episode: 850 | Reward: -25.0 | Epsilon: 0.050 | Steps: 25
Episode: 900 | Reward: -25.0 | Epsilon: 0.050 | Steps: 25
Episode: 950 | Reward: -25.0 | Epsilon: 0.050 | Steps: 25
```

上圖為 `double_DQN.py` 在 **Player Mode** 下的訓練進度，顯示雙網路架構在隨機起點環境中的學習軌跡與收斂情況。

Made with GAMMA

第三章 · HW3-3

Lightning 框架與訓練優化技巧

PyTorch Lightning 效益

- **程式碼解耦**：訓練邏輯（**Trainer**）與模型邏輯（**Module**）分離
- **硬體加速**：自動偵測 **RTX 4090**，**2000** 步訓練僅耗時 **12** 秒

進階穩定技巧

1. **Huber Loss**：對離群值具魯棒性，穩定梯度
2. **Soft Target Update**：以 ***Itau*** 參數每步微調，**Q** 值更新更平滑
3. **Gradient Clipping**：限制梯度範數為 **1.0**，防止梯度爆炸
4. **LR Scheduler**：階梯式調降學習率，助後期收斂

Train Loss

0.105（穩定收斂）

訓練效率

163.05 it/s

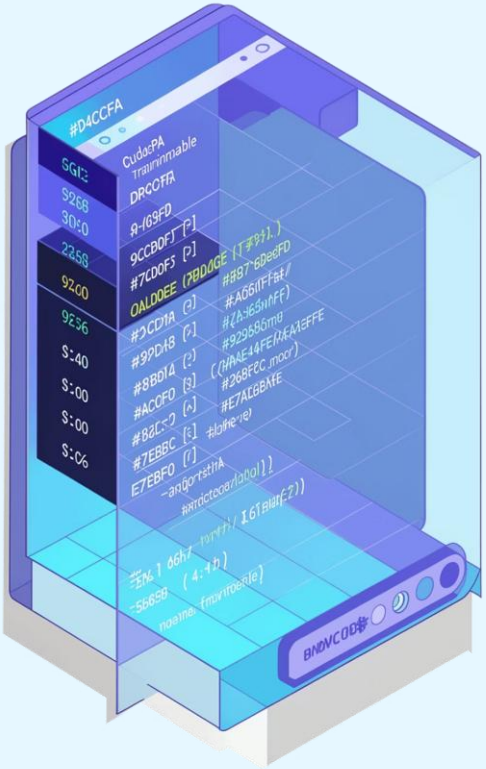
GPU 狀態

CUDA 已啟用

最終 Epsilon

0.135

Made with GAMMA

```
Epoch 0: 100%|                                        | 2000/2000 [00:12<00:00, 163.05it/s, v_num=2, train_loss=0.105, epsilon=0.135]
Trainer.fit` stopped: `max_epochs=1` reached.
Epoch 0: 100%|                                        | 2000/2000 [00:12<00:00, 162.96it/s, v_num=2, train_loss=0.105, epsilon=0.135]
```

上圖為 `light_DQN.py` 在 **Random Mode** 下的最終執行結果，重點呈現 `GPU available: True (cuda)`、`train_loss=0.105`，以及 `epsilon=0.135`，證實模型已穩定收斂。